Devups

Tome IV — Les modules fournis

Documentation du framework

Deuxième édition — novembre 2026 · Devups 7.1.0

Établie à partir du code du dépôt **spacekola/devups** (github.com/yuricoco/devups-package), version 7.0.0, et de sa suite de tests.

Sommaire

Chapitre 1 — Les composants du socle 4

1.1 La règle 4

1.2 `devups` — le noyau d'administration 4

1.3 `devupscms` — le contenu et la communication 4

1.4 `devupsfront` — le côté public 4

Chapitre 2 — Les rôles et les droits 6

2.1 Le modèle 6

2.2 Le catalogue : `rights:sync` 6

2.3 L'attribution 6

2.4 `can()` 7

2.5 Le cache compilé 7

2.6 Où les droits sont vérifiés 7

Chapitre 3 — Le journal 8

3.1 L'API 8

3.2 Le regroupement par empreinte 8

3.3 Le gestionnaire global 9

3.4 L'écran 9

Chapitre 4 — Le CMS 10

4.1 Le modèle 10

4.2 L'édition 10

4.3 Lire un contenu 10

Chapitre 5 — `ModuleTree` : les données de référence 11

5.1 Le problème qu'il résout 11

5.2 Le modèle 11

5.3 `branches` — le schéma de profondeur 12

5.4 Les quatre modes de liaison 12

5.5 Le contrat de lecture 12

5.6 Les contraintes du modèle 13

5.7 `chain` est libre 13

Chapitre 6 — Le protocole de cache, les statuts, les pays 14

6.1 Pourquoi un protocole 14

6.2 Les sept règles 14

6.3 `Status` — décrire, ne pas remplacer 15

6.4 `Interval` et `Interval_group` 15

6.5 `Country` 15

Chapitre 7 — Notifications, Web Push et file de tâches 16

7.1 Une seule entité, deux destinataires 16

7.2 Lecture 16
7.3 Web Push 16
7.4 La file de tâches 17

Chapitre 8 — Courriels : modèles et journal 18

8.1 `DMail` 18
8.2 `Reportingmodel` 18
8.3 `Emaillog` 18

Chapitre 9 — `ModuleUser` : les utilisateurs 20

9.1 L'entité 20
9.2 Les routes 20
9.3 `Configuration` 20

Chapitre 10 — Les dettes connues du framework 22

10.1 Ce que la 7.0 et la 7.1 ont soldé 22
10.2 Les dettes ouvertes 23
10.3 Les leçons de méthode 23

Chapitre 11 — Récapitulatif pour qui reprend un projet 25

11.1 Les dix premières minutes 25
11.2 Les dix pièges qui coûtent le plus cher 25
11.3 Les commandes du quotidien 26
11.4 Comment contribuer à cette documentation 26

Chapitre 1 — Les composants du socle

1.1 La règle

Avant de coder un mécanisme, vérifier qu'un module du socle ne le fait pas déjà. Les droits, le journal, les langues, les contenus éditables, les listes de référence, les statuts, les notifications et les courriels ont chacun leur module : les réinventer dans un projet, c'est perdre l'écran d'administration, la traduction, le cache et les tests qui vont avec.

1.2 `devups` — le noyau d'administration

Module	Entités	Rôle
`ModuleAdmin`	`Dvups_admin`, `Dvups_role`, `Dvups_action`, `Dvups_privilege`	comptes d'administration, rôles et droits (chapitre 2)
`ModuleConfig`	`Configuration`, `Dvups_entity`	le registre de paramètres, le catalogue des entités
`ModuleLang`	`Dvups_lang`	les langues (Tome III, chapitre 7)
`ModuleLog`	`Dvups_log`	le journal applicatif (chapitre 3)

1.3 `devupscms` — le contenu et la communication

Module	Entités	Rôle
`ModuleCms`	`Cmscontent`, `Cmsresource`, `Cmscomment`	contenus éditables et traduits, leurs images, leurs commentaires (chapitre 4)
`ModuleTree`	`Tree`, `Tree_item`, `Tree_item_image`	les données de référence en arbre (chapitre 5)
`ModuleData`	`Status`, `Interval`, `Interval_group`	statuts, intervalles et barèmes (chapitre 6)
`ModuleLocation`	`Country`	les pays, traduits (chapitre 6)
`ModuleNotification`	`Notification`, `Notificationtype`	notifications aux utilisateurs et aux administrateurs (chapitre 7)
`ModulePushNotification`	`Push_subscription`, `Workers`	Web Push, FCM, file de tâches (chapitre 7)
`ModuleReporting`	`Reportingmodel`, `Emaillog`, `Push_email`	modèles de courriel et journal des envois (chapitre 8)

1.4 `devupsfront` — le côté public

Module	Entités	Rôle
`ModuleUser`	`User`, `Oauth_client`	inscription, connexion, profils, Google et Apple (chapitre 9)

Retirés en 7.0. `ModuleGallery` (`Dv_image`) — remplacé par `ImageCore` porté directement par `Cmsre-source` et `Tree_item_image` ; `ModuleDatabase` (`request_history`), jamais alimenté ; les entités de traduction en base `Local_content*` et `Page_mapped` ; dans `ModuleLocation` , `Continent` , `Region` , `Department` , `District` et `Town` ; `Notificationbroadcasted` ; `Dvups_right` , remplacé par `Dvups_action` et `Dvups_privilege` . Les migrations correspondantes sont livrées dans `database/migration/` .

Chapitre 2 — Les rôles et les droits

> **Nouveau en 7.0.** Le système de droits a été réécrit : un catalogue d'actions par entité, des privilèges par rôle, une fonction unique `can()`, un cache compilé.

2.1 Le modèle

Table	Contenu
`dvups_admin`	un compte d'administration, rattaché à un rôle
`dvups_role`	un rôle (`name` , `alias`) et son menu matérialisé
`dvups_action`	le catalogue : une ligne par couple (entité, action), avec un libellé
`dvups_privilege`	un couple (rôle, action) : ce rôle a le droit de faire cette action

Toute entité a six actions implicites — `create`, `read`, `update`, `delete`, `export`, `import` — plus les actions métier déclarées dans la clé `actions` de son datacore (Tome II, §1.4). S'y ajoute une action « virtuelle », sans entité : `Documentation/read`, qui ouvre le Swagger du back-office.

2.2 Le catalogue : `rights:sync`

```
php devups rights:sync
```

`Dvups_action::sync()` parcourt `config/dvups_configurations.php` et le datacore de chaque entité, et crée les lignes `dvups_action` manquantes. Il ne supprime ni ne modifie jamais une ligne existante : une action retirée d'un datacore reste en base avec ses privilèges, à l'administrateur de la retirer explicitement. La commande rafraîchit ensuite le menu matérialisé de chaque rôle, pour qu'une entité nouvelle ou retirée apparaisse ou disparaisse de la barre latérale sans attendre la prochaine sauvegarde du rôle.

À lancer après tout `dvups_:update`.

2.3 L'attribution

L'écran « Rôles » du back-office présente, sur le formulaire même du rôle, une grille entité × action de cases à cocher. Enregistrer le rôle réécrit ses privilèges et vide son cache.

> **Le rôle `admin` est le rôle maître.** `can()` le laisse toujours passer, quels que soient ses privilèges. Et **seul ce rôle** peut modifier les privilèges d'un rôle ou changer le rôle d'un administrateur — même si un autre rôle a reçu `update` sur `Dvups_role` ou `Dvups_admin` pour d'autres usages (renommer un rôle, modifier un profil). Sans cette règle, avoir `update` sur les rôles reviendrait à avoir tous les droits.

2.4 `can()`

```
if (!can('close', Task::class)) {
    http_response_code(403);
    return ['success' => false, 'detail' => 'forbidden'];
}
```

`can(string $action, string $entité): bool` :

1. refuse s'il n'y a pas d'administrateur en session ;
2. relit l'administrateur courant en base (une requête par processus, mise en cache) : un changement de rôle est pris en compte **dès la requête suivante, sans reconnexion** — `$_SESSION[ADMIN]` n'est qu'un instantané pris à la connexion ;
3. laisse passer le rôle `admin` ;
4. sinon, consulte les droits compilés du rôle.

Le nom d'entité peut être pleinement qualifié : seul le nom court est retenu.

2.5 Le cache compilé

`Dvups_role::compiledRights($roleId)` construit, une fois par rôle, un tableau `[entité][action] => true` à partir de `dvups_privilege`, et l'écrit dans `cache/rights/<roleId>.php`. `Dvups_role::clearRightsCache($roleId)` l'invalide : à chaque modification des privilèges du rôle et à sa suppression. Une page de liste qui appelle `can()` trois fois par ligne ne coûte donc qu'une lecture de fichier.

2.6 Où les droits sont vérifiés

Endroit	Ce qui est vérifié
`Request::Controller()`	toute action CRUD d'entité, par une page ou par `admin/api/<entité>/<action>`
`Dvups_roleController`, `Dvups_adminController`	chacune de leurs méthodes, y compris par les routes historiques `services.php?path=...`
`Datatable`	l'affichage des boutons voir, modifier, supprimer
`/admin/apidocs`	`can('read', 'Documentation')`
une action métier	à la main, dans le contrôleur

Chapitre 3 — Le journal

> **Nouveau en 7.0.** `dvups_log` était une table fourre-tout, et `email_log` servait aussi à consigner des erreurs. Le journal est désormais un module dédié, avec une API unique et un gestionnaire global d'erreurs.

3.1 L'API

```
Log::error($messageOuThrowable, $contexte = []);
Log::warning(...);
Log::info(...);
```

Utilisable partout : noyau, front, back-office, CLI. Elle n'interrompt jamais l'appelant : une erreur d'écriture du journal, en base ou sur disque, est elle-même avalée.

```
try {
    $client->send($payload);
} catch (\Throwable $e) {
    Log::error($e, ['task_id' => $task->getId()]);
    return ['success' => false, 'detail' => t('Envoi impossible pour le moment')];
}
```

3.2 Le regroupement par empreinte

Les occurrences identiques sont regroupées en une seule ligne. L'empreinte (`sha1`) est calculée sur la classe, le fichier, la ligne et le message normalisé — les nombres variables, comme un identifiant, sont fusionnés. La ligne garde un compteur d'occurrences (incrémenté de façon atomique), la date de première et de dernière occurrence, le niveau, le contexte.

Colonne	Contenu
`fingerprint`	l'empreinte
`level`	`error`, `warning`, `info`
`entity_class`, `file`, `line`	l'origine
`message`, `context`	le message (aperçu) et le contexte JSON
`occurrences`, `first_at`, `last_at`	le regroupement
`log_file`	le fichier de débordement, s'il existe

Un message ou une trace de plus de 1 000 caractères (`Log::INLINE_MAX_LENGTH`) débordent vers `cache/log/<AAAA-MM>/<empreinte>.log`, non versionné, écrasé à chaque nouvelle occurrence. La base ne garde que le chemin et un aperçu.

`Log::purge()` supprime les lignes, et leurs fichiers, plus anciennes que `LOG_RETENTION_DAYS`. Elle n'est jamais appelée automatiquement : la planifier dans `cron.php`.

3.3 Le gestionnaire global

`Log::installGlobalHandler()`, appelé une seule fois par `header.php` et `admin/header.php`, installe `set_exception_handler`, `set_error_handler` et une fonction d'arrêt pour les erreurs fatales.

- Le fichier et la ligne enregistrés sont ceux que PHP transmet : l'erreur est attribuée à son vrai point d'origine.
- `error_reporting()` est respecté : une erreur masquée par `@` n'est jamais enregistrée.
- Seuls les niveaux supérieurs ou égaux à `LOG_MIN_LEVEL` (`warning` par défaut) sont écrits : `E_NOTICE` et `E_DEPRECATED` sont ignorés par défaut, sans empêcher PHP de les traiter par ailleurs.
- Une garde contre la récursion empêche une erreur survenue pendant l'écriture du journal de reboucler.
- En production, la réponse est générique : une page, du JSON selon le contexte, ou du texte brut en CLI. En développement, le détail est affiché.

Le gestionnaire n'est pas installé par `dependanceInjection.php`, partagé avec la CLI et PHPUnit, dont il perturberait la gestion d'erreurs.

3.4 L'écran

L'écran « Logs » du back-office est en lecture seule : liste, filtres par niveau et par date, fiche de détail qui affiche le fichier de débordement complet. Il est protégé par `can('read', Dvups_log::class)`.

> **Journal et courriels sont séparés.** `Emaillog` ne consigne que les courriels réellement envoyés (chapitre 8). Une erreur, même liée à un envoi, va dans `Log::`.

Chapitre 4 — Le CMS

> **Renommé en 7.0.** `Cmstext` devient `Cmscontent`, `Imagecms` devient `Cmsresource`, et `Cmscomment` est généré pour la première fois. La migration `2026_11_08_phase4_cms_rename.sql` renomme les tables en conservant les données, puis `…_phase4_drop_old_images.php` ne supprime les anciennes tables d'images que si la reprise est vérifiée complète.

4.1 Le modèle

Entité	Rôle	Colonnes principales
`Cmscontent`	un contenu éditable, traduit, rattaché à un élément d'arbre	`reference`, `slug`, `active`, `position` ; traduites : titre, contenu
`Cmsresource`	une image ou un fichier d'un contenu ; étend `ImageCore` et porte elle-même son fichier	colonnes d'`ImageCore`, relation vers `Cmscontent`
`Cmscomment`	un commentaire sur un contenu, lié à un `User`	`pseudo`, `email`, `comment`, `seen`

`Cmscontent` est rattaché à un `Tree_item` : la page ou la rubrique à laquelle il appartient. C'est ce qui permet de construire un menu (un arbre) et d'y accrocher des contenus sans table supplémentaire.

4.2 L'édition

La page `/admin/devupscms/ModuleCms/cmscontent/edit?id=<id>` édite un contenu dans toutes ses langues et liste ses ressources (`CmsresourceTable`). Le téléversement passe par `Cmsresource::uploadImage()`, qui s'appuie sur `Dfile`.

4.3 Lire un contenu

```
$about = Cmscontent::where('this.slug', '=', 'a-propos')
    ->setLang($idLang)
    ->firstOrNull();
```

Lire par `slug` (clé machine), jamais par titre (libellé traduit) ; forcer la langue reste utile pour obtenir une chaîne simple plutôt que le format bilingue, même si l'ancien défaut D1 (Tome II §7.4) ne rend plus la ligne invisible depuis 7.1.

Chapitre 5 — `ModuleTree` : les données de référence

5.1 Le problème qu'il résout

Toute application a des données meubles : des valeurs qui remplissent des listes déroulantes et changent rarement. Des catégories, des types, des critères, des liens de réseaux sociaux, un menu. Chacune, prise seule, appelle une table : un `id`, un libellé, parfois un parent, une position. Toujours les mêmes colonnes.

`ModuleTree` les remplace par une structure générique en arbre, et compile chaque arbre en fichiers PHP pour un accès de type constante clé-valeur. Le gain n'est pas d'économiser des tables : c'est d'avoir un seul écran d'administration, un seul mécanisme de traduction et un seul cache pour toutes les énumérations du projet.

5.2 Le modèle

`tree` — un arbre

Colonne	Note
`name`	la clé machine — unique, et **immuable** une fois utilisée dans le code
`branches`	les noms de paliers, séparés par des virgules — pilote la profondeur (§5.3)
`description`	

`tree_item` — un élément

Colonne	Note
`tree_id`	l'arbre
`slug`	la clé machine — unique par arbre
`position`	l'ordre d'affichage
`parent_id`	auto-adjacence
`main`	1 = élément de premier niveau
`status`	
`chain`	libre — chaque usage définit son format (§5.7)

Plus `tree_lang` (`label`) et `tree_item_lang` (`name`) pour les libellés, et `Tree_item_image` (qui étend `Image-Core`) pour les images d'un élément.

Le code se réfère toujours à la clé machine (`tree.name` , `tree_item.slug`), jamais au libellé. C'est ce qui rend le cache clé-valeur possible.

5.3 `branches` — le schéma de profondeur

`tree.branches` est la liste, du plus haut au plus bas, des noms de paliers de l'arbre. Une hiérarchie anonyme — « un élément, ses enfants, leurs enfants » — ne dit rien de ce qu'est chaque niveau ; nommer les niveaux leur donne une identité. Un arbre `catalogue` peut déclarer `famille,sous-famille,gamme`.

- Le gestionnaire d'arbre refuse de créer un sous-élément au-delà de la profondeur déclarée.
- Le cache descend sur `count(branches)` niveaux.
- Chaque entrée du cache porte son `level` (1 = racine) et le nom de son palier (`branch`).
- Un arbre sans `branches` garde un unique niveau.

`Tree::setBranches()` retire les espaces de bordure et refuse explicitement tout segment vide (virgule en double ou en bordure), en retournant un message d'erreur comme tout setter de validation.

5.4 Les quatre modes de liaison

Une entité qui consomme une donnée de référence peut s'y rattacher de quatre façons.

Mode	Forme	Usage
Relation	`@ManyToOne(targetEntity="\Tree_item")`, `onDelete="set null"`	une valeur. Filtrable, intègre, jointe automatiquement
Table pivot	`<entité>_<référence>`, deux clés étrangères	plusieurs valeurs. Filtrable, intègre
Instantané par slug	la valeur figée dans la ligne à sa création, souvent en JSON	enregistrements historiques qui doivent garder la valeur d'origine même si elle est renommée ensuite ; impose l'immuabilité du slug
Chaîne dénormalisée	`"3,7,12"` dans une colonne texte	**proscrit** : aucune intégrité, aucun filtrage SQL possible

> **La règle.** Tout nouveau consommateur utilise une relation (une valeur) ou une table pivot (plusieurs valeurs). L'instantané est réservé aux enregistrements historiques. La chaîne dénormalisée ne se révèle comme dette que le jour où il faut filtrer — et ce jour-là, `?categories.id:eq=3` échoue.

5.5 Le contrat de lecture

```
Tree::constant('category');                    // la liste ordonnée, pour une liste déroulante
Tree::constant('category', 'electronique');     // un élément précis, par son slug
Tree::constant('category', null, $idLang);      // dans une langue donnée
```

`Tree::constant(string $name, ?string $slug = null, ?int $id_lang = null)` est le point d'entrée unique de lecture. `Tree::readCache($name)` lui délègue, conservé pour compatibilité.

5.6 Les contraintes du modèle

#	Contrainte	Garantie par
U1	`tree.name` unique, non nul, de la forme `[a-z0-9_]+`	à poser en base par migration (le schéma livré ne la déclare pas)
U2	`tree.name` immuable	`Tree::__update()`, qui compare à la valeur en base et lève une exception
U3	`tree_item.slug` unique par arbre	à poser en base : index unique `(tree_id, slug)`
U4	un `slug` référencé par un instantané ne doit plus changer	discipline du projet
U5	tout élément destiné au cache a un `slug` non nul	discipline du projet ; sinon l'élément est exclu du cache (C7)

U2 fait fonctionner tout le reste : le cache est indexé par `tree.name`. Si le nom pouvait changer, le fichier de cache deviendrait orphelin sans que rien ne le signale.

5.7 `chain` est libre

`chain` stockait autrefois la liste des parents d'un élément ; `branches` l'a remplacée. La colonne est désormais polyvalente : chaque usage définit son format.

> **La leçon est transférable.** Une colonne volontairement polyvalente ne doit pas avoir d'accesseur typé : l'accesseur crée une promesse de format que la colonne ne tient pas.

Chapitre 6 — Le protocole de cache, les statuts, les pays

6.1 Pourquoi un protocole

Compiler des données de référence en fichiers PHP est une idée simple qui peut mal tourner de sept façons. Les sept règles qui suivent ont été établies en construisant le cache de `Tree`, puis revérifiées en le réimplémentant pour `Status`. Elles s'appliquent à tout cache de données de référence.

`Tree::buildCacheData($name, $id_lang)` produit, par langue active, deux fichiers :

Fichier	Forme	Usage
`cache/trees/<name>.<iso>.php`	liste ordonnée	rendu d'une liste déroulante, d'un menu
`cache/trees/<name>_references.<iso>.php`	carte indexée par slug	recherche en temps constant

6.2 Les sept règles

C1 — La clé de fichier. Le nom du fichier dérive d'une clé unique et immuable, et il est assaini à l'écriture. `Util::arrayToPhpFile()` n'accepte que `^[A-Za-z0-9_\-]+(\.[A-Za-z0-9_\-]+)*$` : les séparateurs `<name>.<iso>` passent, `../` non.

C2 — La langue. Un fichier par langue active, et la langue est propagée à chaque niveau de la descente — sinon les enfants restent dans la langue par défaut.

C3 — L'invalidation à l'écriture. Toute écriture (`Tree_item::__insert()`, `__update()`, `__delete()`, `Tree::__delete()`) appelle `Tree::markDirty($name)`. Une seule fonction d'arrêt par processus reconstruit, en fin de requête, chaque arbre marqué : une reconstruction par arbre et par requête, depuis l'état final en base. Conséquence pour les tests : le rebuild différé est invisible dans le processus qui écrit, il se vérifie dans un second processus.

C4 — L'atomicité. Fichier temporaire dans le même dossier, puis `rename()`, exception si l'une des deux étapes échoue. Sans cela, un lecteur concurrent peut inclure un fichier à moitié écrit, donc du PHP invalide.

C5 — La lecture honnête. Un cache absent doit se voir. `Tree::constant()` lit le fichier de la langue demandée, retombe sur la langue par défaut, puis reconstruit depuis la base ; dans ce dernier cas, il journalise toujours, et lève une exception hors production. La lecture renvoie `null` (et non `[]`) quand le fichier est absent : un cache manquant et un arbre vide restent distinguables. Et elle signale par le journal, jamais par `dv_dump()`, qui tuerait la page.

C6 — La profondeur. `maxDepth = min(count(branches), Tree::MAX_CACHE_DEPTH)`, avec `MAX_CACHE_DEPTH = 10`. Un garde anti-cycle (`parent_id` A → B → A) arrête la descente au nœud déjà visité, en journalisant, au lieu de boucler.

C7 — Ce qui n'est pas mis en cache. Le dire explicitement. Pour `Tree` : les éléments sans `slug` et ceux dont `main != 1` sont exclus. Un élément sans `slug` n'apparaît donc jamais dans le cache, silencieusement — une exclusion silencieuse ressemble en tout point à une donnée absente.

Règle	Verdict de la seconde implémentation (`Status`)
C1 — clé de fichier	adaptée : la clé est `entity`, partagée par nature
C2, C3, C4, C5	inchangées
C6 — profondeur	sans objet : les statuts sont plats
C7 — exclusions	adaptée : un statut sans `entity` n'est jamais mis en cache

6.3 `Status` — décrire, ne pas remplacer

Beaucoup d'entités ont une colonne `status` entière. `Status` est une table de **description** : une clé machine (`draft`, `published`…), l'entier qu'elle décrit (`value`), une couleur, une position, un libellé traduit, et une portée (`entity`). Une contrainte `UNIQUE(entity, _key)`, à poser par migration, empêche deux lignes de porter la même clé pour la même entité.

> **Pourquoi la table ne devient pas la source de vérité.** Le code métier compare la colonne `status` à des entiers (`status = 1`), jamais par une jointure vers `status`. Faire dépendre une règle de visibilité d'une table éditable par un administrateur — où l'on peut ajouter, renommer ou supprimer une clé — serait fragile. La table dit ce que les entiers veulent dire pour un humain ; elle ne décide jamais ce qui est visible. Quand on ne sait pas quel entier une clé décrit, laisser `value` à `NULL` : c'est une information ; deviner est une erreur silencieuse.

```
Status::constant('task');               // tous les statuts d'une entité, depuis le cache
Status::constant('task', 'draft');      // un statut précis
Status::constant('task', null, $idLang);
$status->is('draft');                   // test
$status->getColortab();                 // pastille HTML colorée (texte contrasté par luminance)
$status->getValue();                    // l'entier décrit
```

Une clé inexistante ne lève pas : elle renvoie une entité vide (Tome II §4.5), et un filtre construit dessus devient silencieusement faux. Vérifier `getId()`.

6.4 `Interval` et `Interval_group`

Des intervalles et des barèmes, rattachés à des éléments d'arbre (`Interval_group` est le pivot `interval` ↔ `tree_item`). Usage typique : des tranches (d'âge, de montant, de durée) éditables au back-office.

6.5 `Country`

`ModuleLocation` ne garde que `Country` : code ISO à deux et trois lettres, indicatif téléphonique, nom traduit. Les données de référence (environ 195 pays) sont chargées, de façon idempotente, par :

```
php devups location:seed
```

La source est `database/seed/countries.php`. Un projet qui a besoin de régions ou de villes les modélise dans son propre composant, ou dans un arbre (`ModuleTree`) si ce sont des listes de référence.

Chapitre 7 — Notifications, Web Push et file de tâches

7.1 Une seule entité, deux destinataires

`Notification` porte soit un `user_id` (utilisateur du site ou de l'application), soit un `admin_id` (administrateur du back-office). Son type (`Notificationtype`, identifié par `_key`) sert au filtrage et à la redirection.

```
Notification::toUser($user, 'task.assigned', ['task' => $task->name]);
Notification::toAdmin($admin, 'task.closed', ['task' => $task->name]);
Notification::toRole($roleId, 'task.overdue', ['n' => $count]);   // tous les admins d'un rôle
```

Le type est **à la fois** la clé de `Notificationtype` et une clé de traduction : le texte est `t($type, $params)`, donc rédigé dans `lang/<iso>.php` :

```
// lang/fr.php
'task.assigned' => 'La tâche :task vous a été assignée',
```

Un `Notificationtype` absent est créé à la volée.

> **API unifiée en 7.0.** `toUser()`, `toAdmin()`, `toRole()` et `broadcast()` sont l'API publique. `on()` et `braodcast()` (faute historique) restent comme alias dépréciés : ils journalisent un avertissement au premier appel. `Notificationbroadcasted` a été supprimée.

7.2 Lecture

Accès	Rôle
`GET /api/user/notifications` (JWT)	les notifications de l'utilisateur connecté
`PUT /api/user/notified` (JWT)	les marquer lues
cloche du back-office	non lues de l'administrateur ; `?notified=<id>` marque lue
`Notification::unreaded($user)`, `unreadedcount($user)`, `unreadedadmin(...)`, `readed($id)`	en PHP

7.3 Web Push

`Push_subscription` enregistre un abonnement navigateur (`endpoint`, `public_key`, `auth_token`, `content_type`, `user_id`). `Push_subscription::sendWebPush()` envoie une notification à cet abonnement via `minishlink/web-push`, signée avec les clés VAPID de `.env` :

```
php -r "require 'vendor/autoload.php'; print_r(Minishlink\WebPush\VAPID::createVapidKeys());"
```

Un navigateur qui répond `410 Gone` (abonnement expiré) fait supprimer l'abonnement automatiquement. Sans clés VAPID configurées, l'envoi ne fait rien et renvoie `false`. Les envois FCM et Expo restent disponibles (`fcmPushNotification()`, `expoPushNotification()`).

7.4 La file de tâches

`Workers` est une file de tâches différées (`type`, `payload`, `callback`, `timer` en minutes). `WorkersController::work()`, appelé par `cron.php`, traite les tâches en attente dont le délai est écoulé — notamment les envois de notifications poussées.

```
# toutes les minutes, en CLI
* * * * * php /chemin/monprojet/cron.php
# ou par HTTP
curl "https://monprojet.example.com/cron.php?token=<CRON_TOKEN>"
```

Chapitre 8 — Courriels : modèles et journal

8.1 `DMail`

`dclass/lib/DMail.php` envoie un courriel par PHPMailer, avec les paramètres SMTP de `.env`, à partir d'un gabarit Blade de `web/views/mails/` (notation pointée : `mails.welcome` → `web/views/mails/welcome.blade.php`). Chaque envoi, réussi ou non, est consigné dans `Emaillog`.

Les gabarits fournis (`welcome`, `otp`, `email`, `invitation`) sont neutres et traduits par `t_lang()` dans la langue du destinataire.

8.2 `Reportingmodel`

Un `Reportingmodel` décrit un modèle de courriel : nom (le gabarit), objet, description, aperçu du contenu attendu (`payload`), libellés traduits.

Écran ou action	Effet
« Collecter les modèles »	parcourt `web/views/mails/*.blade.php` et crée les `Reportingmodel` manquants ; ne supprime jamais rien
créer un modèle	crée aussi son fichier gabarit s'il n'existe pas ; n'écrase jamais un fichier existant
aperçu	rend le gabarit avec un payload d'exemple, sans envoi
envoi de test	envoie réellement, via `DMail`
`Reportingmodel::sendMail()`	envoi depuis le code, consigné dans `Emaillog`

Le lien entre les deux sens — du fichier vers la base, de la base vers le fichier — fait du dossier `web/views/mails/` et de l'écran « Reporting » deux vues du même catalogue.

> **Le mot de passe SMTP n'est jamais affiché.** L'aide intégrée des écrans de courriel montre l'hôte, le port et l'expéditeur, mais masque le mot de passe.

8.3 `Emaillog`

Colonne	Contenu
`destinataire`, `sujet`, `modele`	l'envoi
`statut`	`envoye` ou `echec`
`erreur`	le message d'erreur, en cas d'échec

```
Emaillog::logSuccess($sujet, $destinataires, 'mails.welcome');
Emaillog::logFailure($sujet, $destinataires, $erreur, 'mails.welcome');
```

`DMail` et `Reportingmodel::sendMail()` appellent ces deux méthodes. L'écran est en lecture seule.

Chapitre 9 — `ModuleUser` : les utilisateurs

9.1 L'entité

`User` étend `UserCore` : identité, `username`, `email`, `phonenumber`, `password` (haché), `lang`, état d'activation, code d'activation et son expiration, jeton de réinitialisation et son expiration, jetons de session.

`User::restrictToPublicProfile()` retire l'e-mail et le téléphone : c'est ce que voit un tiers (Tome III, §8.6).

9.2 Les routes

`UserFrontController` (`@Api(name='/user')`) :

Route	Accès	Rôle
`POST /api/user/registration`	public	inscription ; envoie un code d'activation
`POST /api/user/authentification`	public	connexion ; renvoie un JWT
`POST /api/user/googleAuth`, `/appleAuth`	public	connexion par fournisseur
`POST /api/user/tokenize`	public	renouvellement de jeton
`GET /api/user/:id`	public	profil, public ou complet selon le demandeur
`GET /api/user/me`	JWT	le profil de l'utilisateur connecté
`POST /api/user/activateaccount`, `GET /resentactivationcode`	JWT	activation par code à 6 chiffres
`POST /api/user/initresetpassword`, `/resetpassword`	public	réinitialisation par jeton dédié (Tome III, §8.7)
`POST /api/user/changepassword`	JWT	changement de mot de passe
`GET /checktelephone`, `POST /changetelephone`	JWT	changement de téléphone par code
`DELETE /delete-account`, `/cancel-delete-account`	JWT	suppression du compte, annulable
`GET /notifications`, `PUT /notified`, `GET /alert`	JWT	notifications
`POST /update-push-token`	JWT	jeton de notification poussée

Les liens envoyés par courriel pointent vers l'application front : `Configuration::get('APP_DEEPLINK')` s'il est défini, sinon `FRONT_URL`.

9.3 `Configuration`

Le registre de paramètres du back-office (`ModuleConfig`) :

```
$deeplink = Configuration::get('APP_DEEPLINK', __env_front);
Configuration::set('MAINTENANCE', 0, 'integer', 'Mode maintenance du site');
```

Un paramètre modifiable par un administrateur va dans `Configuration`. Un secret ou une valeur propre à l'environnement va dans `.env`.

Chapitre 10 — Les dettes connues du framework

Un défaut connu et nommé coûte une heure ; le même défaut redécouvert coûte une journée.

10.1 Ce que la 7.0 et la 7.1 ont soldé

Dette des versions 4.x	Correction
`$dvold` posé par référence : les mises à jour n'écrivaient rien	vrai clone dans `hydrate()`
erreur SQL = affichage puis arrêt	`DBALException`, rattrapable
`install` : `config_data.sql` échouait en silence, pas d'administrateur	fichier réduit aux `INSERT`, séquence testée
`services.php` des modules acceptaient des écritures anonymes	session d'administration exigée
profil utilisateur public exposant e-mail et téléphone	`restrictForRequester()`
mots de passe en `sha1`, cookie « se souvenir » contenant le hash	`password_hash`, jeton aléatoire
aucune limitation des tentatives	`RateLimiter`
inclusion de fichier par `?lang=`	`dv_valid_lang()` + `realpath()`
traductions de l'interface en base, une requête par libellé	fichiers `lang/*.php`, aucune requête
`Dvups_right` sans vérification réelle	`can()`, catalogue, privilèges, cache
secrets dans un fichier versionné	`.env`
schéma modifié à la main par environnement	`php devups migrate`
`Datatable` : 500 après toute création ou mise à jour par certains contrôleurs	`catch (\Throwable)` dans le rendu de ligne
D1 : `INNER JOIN dvups_lang` après le `LEFT JOIN <entité>_lang` — une ligne traduite sans traduction disparaissait de toute lecture ORM	`LEFT JOIN` vers `dvups_lang` (7.1)
D2 : colonne ambiguë sur une entité traduite (`l.name`)	qualification automatique `this.<colonne>` quand l'entité est traduite (7.1)
D3 : `get($col, $id_lang)` — second argument sans effet sur la langue	langue posée avant `initSelect()` (7.1)
D4 : `whereIn()` / `in()` / `notIn()` / `whereInReverse()` quotaient sans échapper réellement	valeurs liées en paramètre PDO (7.1)

Dette des versions 4.x	Correction
D5 : `whereBetween()` / `between()` interpolaient leurs bornes	bornes liées en paramètre PDO (7.1)
D6 : `nullable` d'une relation obligatoire ignoré par le générateur	`nullable=false` sur `@JoinColumn` quand la relation est `'not'` (7.1)
D7 : `core:g:crud` pouvait dupliquer le bloc de `require` du module	`__Generator::dependencies()` vérifie l'idempotence avant d'écrire (7.1)
D10 : contrôle de session vide en tête de `manageServe()`	vrai refus 401 JSON, mêmes exceptions que `manage()` (7.1)
D12 : une clé de colonne `Datatable` qui ne nomme aucune propriété arrêtait l'écran (`\TypeError` de `call_user_func()`)	`method_exists()` vérifié avant l'appel : cellule vide + `Log::warning()` (7.1)
D13 (partie `StatusTrait`) : `StatusTrait` référençait une entité disparue	supprimée, aucun usage réel (7.1) — `Dvups_config_item`, déclarée dans la même dette, s'est révélée toujours utilisée (héritée par `Dvups_entity`) : retirée de la liste, ce n'était pas du code mort
D15 : téléversement d'une image de 1×1 pixel — erreur de redimensionnement GD plantant toute la requête	`Dfile` conserve l'original déjà enregistré et renonce proprement à la seule variante redimensionnée en cas d'échec GD (7.1)

10.2 Les dettes ouvertes

#	Dette	Emplacement	État
D8	`schema:update --force` peut répondre « success » sans appliquer un renommage d'index	Doctrine	passer par une migration
D9	`Router::cacheRoute()` commence par `return null;`	`Router.php`	confirmé en 7.1 (audit explicite, ÉTAPE E. 4) : code mort par construction, documenté en commentaire plutôt que supprimé ; laisser `ROUTE_CACHE=false`
D11	`FORMTYPE_INJECT_COLLECTIONFORM` déclaré, sans rendu	`FormManager`, `FormFactory`	marqué `@deprecated` en 7.1 ; à ne pas utiliser
D14	`whereRaw()` et `where_str()` ont un corps identique	`QueryBuilder`	laissé tel quel ; jamais avec une donnée utilisateur

10.3 Les leçons de méthode

- **Mesurer avant de corriger.** Une affirmation sur un comportement se vérifie par un test qui l'exécute, pas par la lecture seule.
- **Corriger à la racine quand elle est dans le projet, la nommer quand elle est dans le socle.** Une correction de `dclass/` change le comportement de tous les projets : elle se fait dans le dépôt du framework, avec un test de non-régression.
- **Un refus doit terminer la requête.** Et ne rien révéler de la ressource demandée.
- **Une exclusion silencieuse ressemble à une donnée absente.** Toute règle qui écarte des lignes (cache, filtre, droit) le dit.

- **Du code mort qui ressemble à de la sécurité est pire que pas de code.** Il fait croire à une protection qui n'existe pas.

Chapitre 11 — Récapitulatif pour qui reprend un projet

11.1 Les dix premières minutes

1. `composer install`, copier `.env.example` en `.env`, renseigner la base.
2. `php devups migrate:status` : la base est-elle à jour ?
3. `composer test` : le projet tient-il ?
4. Lire `src/requires.php` : la liste des modules réellement chargés.
5. Lire `route/WebService.php` : ce qui est exposé (`$public_access`), ce qui est public (`$publicMethods`).
6. Ouvrir le back-office, écran « Rôles » : qui a le droit de faire quoi.
7. Ouvrir l'écran « Logs » : ce qui casse en ce moment.
8. `php devups lang:missing` : ce qui n'est pas traduit.
9. `php devups schema:update --dump-sql` : le schéma réel correspond-il aux entités ?
10. `/admin/apidocs` : la carte de l'API.

11.2 Les dix pièges qui coûtent le plus cher

#	Piège	Référence
1	`find()`, `first()`, `getInstance()` renvoient une entité vide, jamais `null`	Tome II §4.5
2	une relation `manyToOne` ne charge que son `id` sans `with()`	Tome II §4.4
3	une ligne traduite sans traduction était invisible à l'ORM (D1, corrigé en 7.1)	Tome II §7.4
4	`?<relation>_id=X` ne filtre rien ; la réponse est paginée à 30	Tome III §2.1
5	`gte` / `lte` n'existent pas et deviennent un `LIKE`	Tome III §4.7
6	une méthode de `FrontController` sans annotation de verbe est morte	Tome III §2.2
7	une méthode `xxxServe()` sans `@Auth` est anonyme	Tome III §8.3
8	`whereIn()` / `whereBetween()` ne sont pas paramétrés	Tome II §4.2
9	un module sans `*Core.json` est invisible, sans erreur	Tome I §7.3
10	`Genesis::json_encode()` et `dv_dump()` terminent la requête	Tome III §6.2, Tome I §4.5

11.3 Les commandes du quotidien

```
php devups serve                        # développer
composer test                           # vérifier
php devups migrate:status               # où en est la base ?
php devups migrate --dry-run && php devups migrate
php devups dvups_:update && php devups rights:sync    # après un nouveau module ou une nouvelle
entité
php devups lang:scan && php devups lang:missing       # traductions
php devups api:doc                      # instantané OpenAPI
php devups core:g:crud monprojet\ModuleTask\task      # générer
```

11.4 Comment contribuer à cette documentation

Chaque projet qui mesure un comportement non documenté, ou trouve une affirmation fausse, l'ajoute ici, avec le fichier qui porte le comportement et, si possible, le test qui le prouve. Une dette soldée passe du tableau 10.2 au tableau 10.1, dans la même version que sa correction.

Fin du Tome IV et de la documentation de Devups 7.0.0.